实验步骤

- **任务**：在实验四的代码基础上，利用GPU共享内存实现矩阵计算优化。
- **子任务**：
  - 对比相同矩阵规模下的计算时间和效率，验证共享内存的加速效果；
  - 调整不同 `BLOCKSIZE`，对比不同 `BLOCKSIZE` 对运算效率的影响；
  - 利用CUDA自带的矩阵乘法算子 `cublasSgemm` 计算矩阵乘法，对比实验结果。
- **目标**：
  - 学会通过使用共享内存减少全局内存访问，提高数据访问效率；
  - 根据共享内存大小优化矩阵分块大小，提升计算性能。
- **实验代码**： `matrix_mul.cu`

0. 复制以下代码，粘贴到实验四的 `matrix_mul.cu` 文件的第22行：

```
    // Block index
    int bx = blockIdx.x;
    int by = blockIdx.y;

    // Thread index
    int tx = threadIdx.x;
    int ty = threadIdx.y;

    // Index of the first sub-matrix of A processed by the block
    int aBegin = wA * BLOCK_SIZE * by;

    // Index of the last sub-matrix of A processed by the block
    int aEnd   = aBegin + wA - 1;

    // Step size used to iterate through the sub-matrices of A
    int aStep  = BLOCK_SIZE;

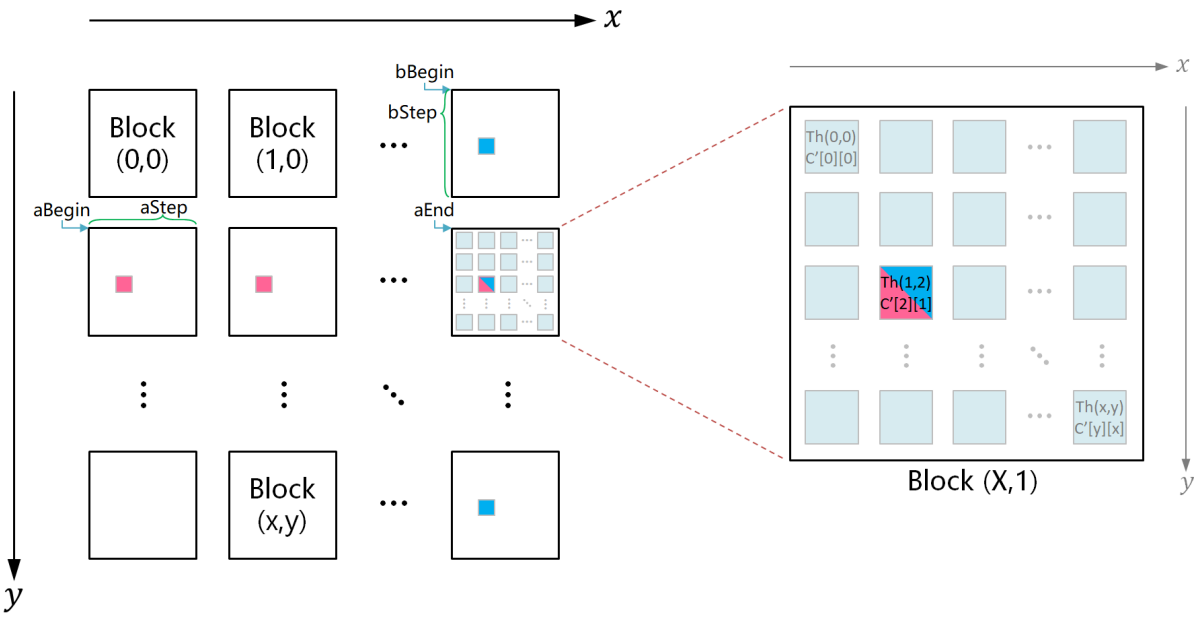
    // Index of the first sub-matrix of B processed by the block
```

Processing math: 100%

```cpp
////////
// Matrix multiplication with shared memory (CUDA Kernel) on the device: C = A * B
////////
const int BLOCK_SIZE = TILE_WIDTH;
__global__ void MatrixMulSharedMemKernel(float *A,
    float *B, float *C, int wA,
    int wB) {


}
```

模板代码的实现思路如下图所示。代码中定义的线程数和矩阵 C 的元素个数相同，每个线程负责计算矩阵 C 的一个元素。每个线程块对应矩阵的一个分块，线程块中的每个线程则对应矩阵分块中的一个元素。



符号说明 📋

上图右侧的 `C'` 表示矩阵 C 对应到图左侧第二行最后一列的线程块中的分块矩阵；`C'[2][1]` 则表示该分块矩阵第3行第2列的元素，其他依此类推。

计算时，每个线程只加载矩阵 A、B 对应到线程块中固定位置的一个元素。例如，上图右侧的线程 `Thread(1,2)` 在第一轮循环只加载图左侧 `aBegin` 指向的分块矩阵中的 `A'[2][1]` 和 `bBegin` 指向的分块矩阵中的 `B'[2][1]`；在第二轮循环只加载 `aBegin + aStep` 指向的分块矩阵中的 `A'[2][1]` 和 `bBegin + bStep` 指向的分块矩阵中的 `B'[2][1]`，依此类推。

尽管线程 `Thread(i,j)` 每轮循环只加载 `A'[j][i]` 和 `B'[j][i]` 两个元素，**但在计算 `C'[j][i]` `[k]` 和 `B'[k][i]` 的所有元素（0 ≤ `k` < `BLOCK_SIZE`）都已经被线程块中的其他线程**

Processing math: 100%

并行加载到了共享存储器中，因此 `Thread(i,j)` 能够正确无误地完成 `C'[j][i]` 的计算。

为方便实现，可将矩阵元素的索引分为 **线程块索引** 与 **块内线程索引** 两级索引。线程块索引是位于线程块左上角的线程的索引值，块内线程索引则是块内线程相对于线程块索引的偏移量。由上图可知，`C'[2][1]` 由第 `(X,1)` 个线程块的第 `(1,2)` 个线程计算，结合矩阵 C 的列数 `N` 和线程块的边长 `BLOCK_SIZE`，即可得到线程块 `(X,1)` 的二维索引是 `[1*BLOCK_SIZE][X*BLOCK_SIZE]`，一维索引是 `(1*N + X)*BLOCK_SIZE`；再由块内线程坐标 `(1,2)` 容易得出块内线程的二维索引是 `[2][1]`，一维索引是 `2*N + 1`。将两级索引值相加得出 `C'[2][1]` 对应到矩阵 C 的全局二维索引是 `[1*BLOCK_SIZE + 2][X*BLOCK_SIZE + 1]`，一维索引是 `(1*N + X)*BLOCK_SIZE + 2*N + 1`。

1. 补全 `MatrixMulSharedMemKernel` 函数的共享内存优化矩阵乘法的代码。在第69行，实现每个线程分别加载矩阵A、B的一个元素到共享内存的 `As[ty][tx]`、`Bs[ty][tx]` 中，注意边界条件判断，避免访问越界导致计算结果有误

```
62      // Load the matrices from device memory
63      // to shared memory; each **thread** loads
64      // one element of each matrix
65      // --- TO DO :Load the elements of the sub-matrix of A into As ---
66      // ---        Load the elements of the sub-matrix of B into Bs ---
67      // NOTE: Ensure that the thread indices do not exceed the matrix dimensions to avoid out-of-bounds access.
68      //       Use boundary checks to load valid elements into shared memory, and set invalid elements to 0.0f
69
```

2. 继续补全实验代码，计算线程负责的子矩阵元素 `Csub` 的累加值（第81行）。使用 `for` 循环遍历共享内存，计算矩阵 `C` 的一个元素的值 —— 将矩阵 `As` 和矩阵 `Bs` 的对应元素相乘并累加到 `Csub` 中

```
79  #pragma unroll
80      // --- TO DO :Implement the matrix multiplication using the sub-matrices As and Bs ---
81
82
83
84
```

3. 将结果写回矩阵 `C`。在第97行，使用块索引（`bx`，`by`）和线程索引（`tx`，`ty`）计算矩阵 `C` 中的全局位置。写入全局内存之前，务必检查全局索引是否在矩阵 `C` 的有效范围内，以避免越界访问

```
93  int c = wB * BLOCK_SIZE * by + BLOCK_SIZE * bx;
94  // --- TO DO :Store the computed Csub result into matrix C ---
95  // NOTE: Ensure that the thread indices "c" do not exceed the matrix dimensions to avoid out-of-bounds access.
96  //       Use boundary checks to write valid elements to the output matrix C.
97
98
99
```

4. 修改 `main` 函数中的 `for` 循环，使其调用 `MatrixMulSharedMemKernel` 函数

```
280     for (int j = 0; j < nIter; j++) {
281       //matrixMulCPU(reference, h_M, h_N, m, k, n);
282       // MatrixMulKernel<<<grid, block>>>(d_M, d_N, d_P, m);
283       MatrixMulSharedMemKernel<<<grid, block>>>(d_M, d_N, d_P, m, n);
284       //cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, n, m, k, &alpha, d_N, n, d_M, k, &beta, d_P, n);
285     }
```

Processing math: 100%

5. 保存文件，执行编译命令脚本，执行可执行文件 `a.out`

```
bash compile.sh
./a.out 1 1000
```

6. 查看并检查结果是否正确，对比使用全局内存的时间开销和计算效率（即与实验四的 `MatrixMulKernel` 函数进行对比）

使用实验室电脑完成本实验时，若计算结果的大部分数值与参考值相差在 0.5 以内，可在上述步骤 3 的结果写回中自行添加四舍五入的逻辑。

7. 更改块大小、矩阵大小（比如改为块大小的整数次幂），对比使用共享内存和全局内存的实验结果

8. 打开第1行的 `USE_CUBLAS` 宏定义，修改 `main` 函数中的 `for` 循环，使其调用 CUDA 自带的矩阵乘法算子，编译运行并对比不同参数（块大小、矩阵大小）下的运行结果差异

```
#define USE_CUBLAS
```

```
for (int j = 0; j < nIter; j++) {
  //matrixMulCPU(reference, h_M, h_N, m, k, n);
  // MatrixMulKernel<<<grid, block>>>(d_M, d_N, d_P, m);
  // MatrixMulSharedMemKernel<<<grid, block>>>(d_M, d_N, d_P, m, n);
  cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, n, m, k, &alpha, d_N, n, d_M, k, &beta, d_P, n);
}
```

`cublasSgemm` 是 CUDA 的 cuBLAS库的矩阵乘法函数，功能是计算 $C = \alpha \cdot A \cdot B + \beta \cdot C$。该函数的使用方法详见 docs.nvidia.com/cuda/cublas/index.html?#cublas-t-gemm。

Processing math: 100%